

Homework 6

① a) $OPT(w)$ = maximum total value of jewellery items that can be placed in a box with a maximum weight capacity of w .

b) Recurrence relation:

$$OPT(w) = \max_{\substack{i=1 \text{ to } n \\ w_i \leq w}} \{ V_i + OPT(w - w_i) \}$$

Base case : $OPT(0) := 0$

The recurrence relation considers all possible items that could be the last one added to the box for a given capacity w .

c) Pseudocode : Using bottom-up approach to fill an array M where $M[w]$ stores the value of $OPT(w)$.

```
UnboundedKnapsack( $w$ , weights, values,  $n$ ):  
   $M$  = array of size  $(w+1)$   
   $M[0] = 0$   
  for  $w$  from 1 to  $W$ :  
     $M[w] = 0$   
    for  $i$  from 1 to  $n$ :  
      if  $weights[i] \leq w$ :
```

$$M[w] = \max(M[w], \text{values}[i] + M[w - \text{weights}[i]])$$

return $M[W]$

- d) Time Complexity: Algorithm has two nested loops.
 Outer loop runs W times.
 Inner loop runs n times.
 $\therefore$ Time complexity: $O(nW)$.

Efficiency: Solution is not efficient.

It is a pseudo-polynomial time algorithm because its runtime depends on the numeric value of the input W , not the size of the input which is $\log_2 W$ bits.

An efficient algorithm must have a runtime that is polynomial in the size of the input.

② a) $OPT(i, k)$ = maximum amount of money that can be won by choosing from the set of briefcases starting from index i to n , given that the total value of chosen briefcases does not exceed a remaining limit of k dollars.

b) Recurrence relation:

Two choices at each briefcase i ;
either take it or don't take it.
Let V_i be the value of briefcase i .

If $V_i > K$:

$$OPT(i, K) = OPT(i+1, K)$$

If $V_i \leq K$:

$$OPT(i, K) = \max(V_i + OPT(i+2, K - V_i), OPT(i+1, K))$$

Base case:

$$OPT(i, K) = 0 \text{ for any } i > n$$

(no. briefcase left)

c) Pseudocode:

Using a 2D array $M[i][k]$, to store the results of the subproblem, filling it from $i=n$ down to 1.

Briefcase Picker (v, k, n):

$M = 2D$ array of size $(n+2) \times (k+1)$

for k from 0 to K :

$M[n+1][k] = 0$

$M[n+2][k] = 0$

for i from n down to 1:

for k from 0 to K :

if $v[i] \leq k$:

$M[i][k] = \max(M[i+1][k],$

$v[i] + M[i+2][k - v[i]])$

else

$M[i][k] = M[i+1][k]$

return $M[1][K]$

d) Time Complexity:

Uses two nested loops:

Outer loop runs n times.

Inner loop runs $k+1$ times.

$\therefore$ Time complexity : $O(nK)$

Efficiency

Solution is not efficient.

It is pseudo-polynomial time because

its runtime is polynomial in the

numeric value of k .

③ a) $OPT(n)$ = maximum profit obtained by cutting up a rod of length n and selling the resulting pieces.

b) Recurrence relation:
Let price of rod i be p_i

$$OPT(n) = \max_{1 \leq i \leq n} (p_i + OPT(n-i))$$

Base case : $OPT(0) = 0$

c) Pseudocode:

RodCutting (P, n):

M = array of size $(N+1)$

$M[0] = 0$

~~for~~ ~~n~~

for n from 1 to N :

$max_val = -infinity$

 for i from 1 to n :

$current_prof = P[i] + M[n-i]$

~~current_prof~~

$max_val = \max(max_val, current_prof)$

$M[n] = max_val$

return $M[N]$

d) Time Complexity :-

It has two nested loops.

Outer loop runs N times.

Inner loop runs N times.

$\therefore$ Total time complexity : $O(N^2)$

Efficiency :

Solution is efficient.

Runtime $O(N^2)$ is polynomial
of the input size N .